Psilocybin Design
A Theory-Method for Deriving Software Organisms from Gaps

Khalid Saqr
ORCID: https://orcid.org/0000-0002-3058-2705

Version 1.0.0
29 June 2026
Technical report MD-TR-2026-001, KNOWDYN LTD

Abstract

Software design often begins after the decisive question has already been compressed into requirements, features, architecture, data models, or stack choices. This paper proposes Psilocybin Design as a theory-method for preserving that prior question. It argues that a software project begins with a gap: a failed or insufficient relation between intention and possibility in a real practice. The method proceeds from the gap to the language-games around the gap, from language-games to hidden verbs, from hidden verbs to relations, from relations to invariants, and from invariants to a software genome. The genome states the minimal identity of the system: what it must sense, remember, transform, connect, protect, expose, repair, or let decay in order to remain itself. Only after this identity is extracted can an organismic model, ontology, survival conditions, and engineering body be justified. Psilocybin Design is not a claim that software is literally alive, nor is it an invitation to decorate software with biological metaphor. It is a disciplined method for deriving software identity before architecture, validating that identity through counter-design and survival tests, and tracing the result into engineering decisions. Its central claim is that software should not first be designed as a tool, product, service, repository, workflow, or stack. It should first be understood by what it must remain in order to continue bridging the gap that called it into existence.

Publication and Rights Notice

1 Introduction: The Prior Question of Software

Software design has powerful ways to reason about structure, decomposition, complexity, domain language, architectural constraint, and implementation. Dijkstra made program structure a matter of disciplined control rather than coding habit [3]. Parnas showed that modularity should be guided by information hiding and design decisions rather than by superficial decomposition [8]. Brooks argued that the essential difficulty of software cannot be removed by any single technical advance

[2]. Evans gave software practice a way to let domain language shape models and boundaries [4]. Fielding showed how architectural style can be derived from environmental constraints in networked systems [5]. Simon situated design within the larger study of artificial things that mediate between inner structure and outer environment [9]. Wittgenstein gave a way to understand language as part of practice rather than as detached naming [12].

These traditions address different levels of software thought. They ask how programs should be structured, how modules should hide design decisions, why complexity persists, how domain language should shape models, how architectural constraints should guide distributed systems, and how artificial systems relate inner organization to outer conditions. Psilocybin Design asks a prior question: before structure, decomposition, domain model, architectural style, or stack can be justified, what identity must the software have?

This prior question matters because software is often designed too late. A condition in the world becomes a requirement. A tension becomes a feature. A recurring failure becomes a ticket. A social or organizational difficulty becomes a workflow. A need for memory becomes storage. A need for trust becomes a dashboard. The translation is useful, but it can also erase the form of the original problem. Once that erasure happens, engineering may proceed with competence while the system is already misidentified.

Psilocybin Design begins before this reduction. It treats a software project as a response to a gap in the world. The task of design is to understand what kind of digital organism must exist in order for that gap to be bridged and for the bridge to survive. The word organism is used with care. The claim is not that software is biologically alive. The claim is that some software must preserve identity under environmental pressure, sense and respond to changes, maintain internal relations, defend boundaries, repair damage, and survive partial failure. In such cases, organismic language can discipline design if, and only if, it constrains what is built.

This paper contributes four things. First, it states a theory of software identity derived from gaps. Second, it provides a genome-extraction protocol that moves from language-game evidence to hidden verbs, relations, invariants, and genome. Third, it validates genome extraction through counter-design and survival tests. Fourth, it requires architectural traceability from genome to invariant, ontology, survival condition, and engineering decision.

The complete movement is:

$$\begin{aligned}\text{Gap} &\to \text{Language-game} \to \text{Hidden Verbs} \to \text{Relations} \\ &\to \text{Invariants} \to \text{Genome} \to \text{Organism} \to \text{Ontology} \\ &\to \text{Survival} \to \text{Stack}.\end{aligned}$$

The notation does not pretend that design has become mathematics. It records dependence. The stack should be the executable body of an identity already discovered, not the premature beginning of the design conversation.

2 Definitions and Scope

Psilocybin Design uses biological language, but it is not a biological theory of software. It is a design theory that borrows organismic reasoning only when identity, adaptation, relation, and survival are central to the software problem. The following definitions fix the terms used throughout the paper.

Term	Definition
Gap	A failed or insufficient relation between intention and possibility. A gap appears when existing arrangements no longer allow meaningful action to continue.
Language-game	The situated words, actions, rules, roles, expectations, workarounds, and judgments through which people live with and speak around the gap.
Hidden verb	An action implied by repeated language around the gap: remember, verify, route, reconcile, expose, authorize, preserve, repair, protect, forget, decay, and similar verbs.
Relation	A dependency, transfer, authority, trace, boundary, memory, transformation, or constraint required by a hidden verb.
Invariant	A condition that must remain true across changing interfaces, data stores, architectures, workflows, or teams.
Genome	The compressed identity of the software: what the system must sense, remember, transform, connect, protect, expose, repair, or let decay in order to remain itself.
Organism	A disciplined organismic model selected because its survival logic fits the genome and constrains design.
Ontology	The internal world of entities, relations, states, and boundaries the organism must recognize in order to preserve its genome.
Survival condition	A testable condition under which the software continues to bridge its original gap.
Death condition	A failure mode in which the system may still run but no longer preserves the identity for which it was created.
Stack	The executable body of the organism: languages, databases, services, APIs, deployment model, tests, observability, documentation, and maintenance practices.

These terms define the scope of the theory. Psilocybin Design is most useful when software must preserve meaning across time, coordinate distributed relations, adapt under pressure, defend identity through change, or remain useful under partial failure. It is less useful for narrow scripts, stable forms, one-off conversions, or simple CRUD systems where the gap is fully expressible as a known function and ordinary engineering methods are sufficient.

The theory should therefore be read as a conditional method. It does not say that every software project is a digital organism. It says that when a software project must preserve identity under environmental pressure, the identity should be extracted before the architecture is chosen. When organismic language does not change the design, tests, ontology, or architecture, it has no value and should be removed.

3 The Gap Before Requirements

A software project begins with a gap. A gap is not merely an absence. It is a break in the relation between what people are trying to do and what the current world allows them to do. Someone needs to remember, coordinate, verify, find, preserve, exchange, constrain, explain, recover, or transform

something, and the present arrangement cannot carry the burden placed upon it.

The gap may be practical, informational, social, technical, organizational, or conceptual. It becomes a software problem when a digital system is called upon to restore, create, or stabilize a possibility of action. This is why the gap is earlier than requirements. A requirement is already a translation. It may be necessary, but it is not innocent. The translation from gap to requirement can lose the identity of the problem.

Consider a team that cannot find past decisions. The immediate request may be search. The gap may not be search. It may be broken memory. A group that distrusts automated recommendations may ask for an explanation panel. The gap may not be explanation. It may be unverifiable authority. An organization that cannot assign work reliably may ask for task management. The gap may not be task assignment. It may be unstable responsibility. A lab that cannot distinguish current findings from obsolete ones may ask for versioning. The gap may not be version control. It may be uncontrolled knowledge decay.

The first discipline of Psilocybin Design is therefore delay. The designer does not move immediately from complaint to feature or from requirement to stack. The designer remains with the gap long enough to ask what relation has failed and what must exist for that relation to become possible again. This delay is not hesitation. It is the protection of identity before construction.

A gap can be studied at four levels.

Level	**Question**	**Example**
Symptomatic gap	What do people first complain about or request?	"We need better search."
Functional gap	What capability appears to be missing?	Locate prior decisions and artifacts.
Relational gap	What relation between intention and possibility has failed?	Decisions no longer remain connected to reasons, evidence, and later work.
Genomic gap	What identity must software have to bridge this failure over time?	A system for preserving and routing recoverable research memory.

The designer should not stop at the symptomatic or functional level. A feature request belongs to the symptomatic surface. A missing capability belongs to the functional layer. Psilocybin Design seeks the relational and genomic layers because they reveal what the software must become, not merely what it should contain.

4 Language-Games as Design Evidence

A gap is not silent. People speak around it before anyone designs a system for it. They ask where something went, who changed it, why it failed, what should happen next, what is allowed, what is missing, and what can be trusted. They create repeated phrases, informal rules, habits, complaints, explanations, and workarounds. These expressions are not noise around the problem. They are evidence of its shape.

The term language-game is useful because it prevents language from being treated as detached

description. Words gain meaning through use. A phrase such as "the latest version," "the source of truth," "who owns this," or "can this be trusted" carries rules, expectations, roles, permissions, and possible failures. The language is already organized by action.

A team that repeatedly asks where a decision was made does not only need storage. It may need recoverable memory. A user who asks whether a result can be trusted does not only need output. The system may need provenance, explanation, and restraint. A group that repeatedly asks who owns a task may not only need assignment. It may need a model of responsibility that survives handoff, absence, and changing dependencies.

Psilocybin Design treats these repeated expressions as the beginning of design knowledge. The designer listens for the actions hidden inside the language. What must the software notice? What must it remember? What must it connect? What must it prevent? What must remain stable when everything around it changes? The language of the gap begins to reveal the identity of the software.

The following table illustrates the extraction pattern.

Observed phrase	**Hidden verb**	**Implied relation**	**Candidate invariant**
"Where was this decided?"	Recover	Decision to reason to artifact to time	Decisions must remain recoverable with rationale.
"Can this result be trusted?"	Verify	Output to evidence to authority to confidence	Claims must carry provenance and confidence state.
"Who owns this now?"	Transfer	Task to authority to continuity	Responsibility must survive handoff.
"Which version is current?"	Distinguish	Artifact to state to time	Currency must be visible and inspectable.
"Why did this change?"	Explain	Change to cause to actor to consequence	Changes must preserve reason and effect.
"What does this depend on?"	Relate	Artifact to dependency to risk	Critical dependencies must be explicit.
"This keeps coming back."	Repair	Failure to recurrence to unresolved cause	Recurrent failures must become design signals.
"We forgot why we rejected that."	Preserve	Rejection to rationale to future reuse	Negative decisions must remain part of memory.

This table should not be read as a fixed taxonomy. It is a discipline of attention. The important move is from nouns to verbs. Many software discussions begin with nouns: dashboard, repository, agent, workflow, database, model, API. Psilocybin Design listens for verbs because verbs reveal survival. A system whose hidden verb is verify has a different genome from one whose hidden verb is remember. A system whose hidden verb is reconcile has a different ontology from one whose

hidden verb is route. A system whose hidden verb is defend has different death conditions from one whose hidden verb is expose.

5 The Genome-Extraction Protocol

The central method of Psilocybin Design is genome extraction. It is the procedure that prevents the theory from becoming metaphor. The genome is not invented by the designer. It is extracted from the gap through evidence in language and practice, then validated against plausible alternatives.

The protocol has five primary stages:

$$\text{Language-game} \rightarrow \text{Hidden Verbs} \rightarrow \text{Relations} \rightarrow \text{Invariants} \rightarrow \text{Genome}.$$

5.1 Stage 1: Collect language around the gap

The designer collects repeated questions, complaints, workarounds, informal names, permissions, failure explanations, and local rules. The evidence should include not only what people ask for, but what they repeatedly repair, distrust, route around, forget, defend, or reconstruct.

Useful sources include interviews, support tickets, issue threads, meeting notes, chat channels, commit messages, operational incidents, onboarding questions, spreadsheet columns, naming conventions, undocumented rituals, and exception-handling practices. The goal is not ethnography for its own sake. The goal is to find the language in which the gap already expresses its structure.

5.2 Stage 2: Extract hidden verbs

The designer translates repeated language into hidden verbs. A hidden verb is an action the software may need to perform or preserve, even if users never request it as a feature. Examples include remember, route, verify, reconcile, authorize, expose, preserve, repair, constrain, decay, forget, defend, amplify, compress, and recover.

A hidden verb should be accepted only if it explains repeated evidence. If the verb appears only because the designer likes it, it should be rejected. If the same evidence can be explained by several verbs, the designer should keep the alternatives and test them later through counter-design.

5.3 Stage 3: Identify relations

A verb implies relations. To remember is not just to store; it is to preserve a relation between event, context, time, actor, and later retrieval. To trust is not just to display confidence; it is to relate claim, evidence, authority, uncertainty, and consequence. To coordinate is not just to send messages; it is to relate actors, tasks, timing, dependencies, and responsibility.

The designer should ask: what must be connected for this verb to be real? What must be kept apart? What must be ordered? What must be transferred? What must be visible? What must be protected? What must be allowed to decay?

5.4 Stage 4: Form invariants

Relations become invariants when they state conditions that must remain true across changes in interface, architecture, workflow, team, and stack. An invariant is not a user story. It is a continuity condition. If the invariant fails, the system may still run but has begun to lose its identity.

For example, if the hidden verb is recover and the relation is decision to rationale to evidence, the invariant may be: every consequential decision must remain recoverable with its rationale and supporting artifacts. If the hidden verb is verify and the relation is claim to source to confidence, the invariant may be: every consequential claim must expose provenance and confidence state.

5.5 Stage 5: Compress into genome

The designer compresses the most important invariants into a genome statement. The statement should be short enough to guide design and strong enough to reject wrong designs. It should not list features. It should state what the software essentially is.

A useful template is:

> This software is essentially a system for *[core capacity]* that must *[hidden verbs]* across *[relations]* so that *[gap bridged]* remains possible under *[environmental pressures]*.

A weak genome says: "This is a searchable archive." A stronger genome says: "This is a system for preserving and routing recoverable research memory across distributed artifacts, decisions, and contributors so that knowledge remains actionable despite time, tool fragmentation, and personnel change."

A weak genome says: "This is a task manager." A stronger genome says: "This is a system for maintaining responsibility across changing actors, dependencies, and states so that work remains accountable despite handoff and uncertainty."

A weak genome says: "This is an AI dashboard." A stronger genome says: "This is a system for making automated claims inspectable, contestable, and accountable so that people can act under uncertainty without surrendering judgment to opaque output."

5.6 Rejection rules

Genome extraction must reject several common false genomes.

- A feature category is not a genome: search, chat, dashboard, repository, workflow, agent, and archive are possible bodies, not identities.
- A stack is not a genome: React, PostgreSQL, vector search, Kubernetes, or a graph database may embody a genome, but none of them states the system's identity.
- A metaphor is not a genome: psilocybin, immune system, nervous system, swarm, membrane, or seed can guide design only after identity has been extracted.
- A slogan is not a genome: "make teams smarter" or "accelerate research" may express ambition but cannot yet reject wrong architecture.

- A domain noun is not a genome: customer, invoice, document, model, project, and task are entities, not identity conditions.

The genome becomes usable only when it can explain why one design fits the gap and another fails it.

6 From Genome to Organism

The organism is selected after the genome, not before it. This ordering is essential. If the designer chooses psilocybin, immune system, nervous system, swarm, membrane, or ecosystem because the image is attractive, the biological language becomes decoration. If the organism is selected because its survival logic fits the genome, the analogy can guide structure, behavior, adaptation, and failure analysis.

An organismic model is justified only when it satisfies five conditions.

1. **Fit:** the organism's survival logic matches the extracted genome.
2. **Constraint:** the analogy changes what is built, tested, or rejected.
3. **Failure:** the analogy exposes ways the system can be damaged or die.
4. **Ontology:** the analogy clarifies what the system must recognize as real.
5. **Discardability:** the analogy can be abandoned when it stops constraining useful design.

Psilocybin is appropriate when the genome requires distributed memory, hidden connectivity, local sensing, signal routing, resilience without a single visible center, and recovery through relation rather than command. Psilocybin is inappropriate when the system's identity is centralized control, narrow calculation, stable record keeping, or one-way publication. In those cases other models, or no organismic model at all, may be better.

Other organismic models can be useful, but they should be treated as disciplined analogies rather than a menu of aesthetic themes. An immune-system model may fit software whose genome is detection, discrimination, response, tolerance, and memory of threats. A nervous-system model may fit software whose genome is sensing, integration, feedback, and coordinated action. A membrane model may fit software whose genome concerns boundary, exchange, filtration, and selective permeability. A seed model may fit software whose identity is dormant potential, self-contained instruction, and context-triggered growth. A wound-healing model may fit software whose identity is repair after damage and restoration of continuity.

The organism should never be asked to do mystical work. It should answer concrete design questions. What counts as a signal? What counts as damage? What must be sensed locally? What must be coordinated globally? What should be centralized? What should remain distributed? What should be forgotten? What should be defended? What should be allowed to mutate? If the organism cannot help answer such questions, it should be removed from the theory of that project.

7 Ontology as the Internal World of the Organism

The genome states identity. The organism gives identity a survival logic. Ontology defines what exists inside that logic. In Psilocybin Design, ontology is not merely a data model. It is the internal world the software must recognize in order to preserve its genome.

An entity belongs in the ontology only if the organism must recognize it to remain itself. A relation belongs only if losing it would damage the genome. A state belongs only if the organism must distinguish it in order to survive. This rule protects the ontology from inflation.

Ontology has three layers.

Layer	**Question**	**Examples**
Entities	What kinds of things must the organism recognize?	Contributor, artifact, claim, decision, task, signal, repository, source, assumption.
Relations	What must be connected, separated, ordered, transferred, or constrained?	Derived from, contradicts, supersedes, owns, depends on, belongs to, routes to, authorizes.
States	What conditions must the organism distinguish?	Fresh, stale, disputed, trusted, orphaned, active, dormant, unresolved, obsolete, decayed.

A weak ontology names objects because they appear in the domain. A strong ontology names entities, relations, and states because the organism cannot survive without them. If the genome is recoverable research memory, then decisions, reasons, claims, evidence, contributors, artifacts, versions, confidence, contradiction, and decay are not optional labels. They are necessary parts of the internal world. If the genome is responsibility across handoff, then ownership, delegation, acceptance, dependency, blockage, authority, and transfer are not administrative details. They are survival elements.

The ontology should also define boundaries. A boundary is not merely a permission rule. It is a distinction the organism must preserve. It may separate public and private knowledge, evidence and speculation, current and obsolete claims, accepted and contested decisions, internal state and external signal, or user authority and system suggestion. Boundary failure is often organism failure.

8 Survival Conditions and Death Conditions

A system can be available and still dead in the sense relevant to Psilocybin Design. It may respond to requests, render pages, and pass ordinary uptime checks while no longer bridging the gap that justified it. Survival is therefore not mere operation. Survival is identity under pressure.

A survival condition states what must remain true for the organism to continue living as itself. A death condition states how the system can lose identity even if the software still runs. Every survival condition should guide at least one engineering implication: a test, monitor, review practice, architectural constraint, data structure, interface affordance, or maintenance obligation.

Survival condition	**Death condition**	**Engineering implication**

Decisions remain recoverable with rationale.	The system stores artifacts but loses why decisions were made.	Decision records, rationale links, author and time metadata, recovery views.
Claims expose provenance and confidence.	Outputs appear authoritative without inspectable grounds.	Provenance graph, evidence links, confidence states, contestation workflow.
Relations remain intact across tools.	Context fragments into disconnected documents, messages, and repositories.	Durable identifiers, graph integrity checks, ingestion from multiple channels.
Stale knowledge becomes visible.	Obsolete claims remain indistinguishable from current knowledge.	Decay signals, review timestamps, freshness indicators, archival states.
Contradictions surface before action is damaged.	Conflicting claims remain hidden until they cause failure.	Conflict detection, contradiction states, review queues, resolution records.
Responsibility survives handoff.	Work loses ownership when people, teams, or dependencies change.	Handoff protocols, acceptance states, owner history, escalation paths.
Loss of one channel does not destroy memory.	Slack, email, meeting notes, or repository loss breaks continuity.	Redundant ingestion, archival policy, exportable records, source reconciliation.
The system can explain its own boundaries.	Users cannot tell what the system knows, ignores, protects, or guesses.	Boundary documentation, permission visibility, uncertainty displays.

Death conditions are especially important because they keep the theory honest. Without them, survival becomes a flattering word. A system whose genome is memory dies when it stores data but loses recoverability. A system whose genome is trust dies when it produces accurate answers that cannot be inspected in consequential contexts. A system whose genome is coordination dies when it sends notifications but cannot maintain responsibility. A system whose genome is adaptation dies when configuration exists but no meaningful environmental change can be sensed.

Survival tests should be stated before the stack is selected. The stack is then evaluated by whether it can embody and protect the survival conditions. This reverses a common practice in which technologies are selected first and the system's survival is later constrained by the accidental properties of those technologies.

9 Validation by Counter-Design and Traceability

A proposed genome is not valid merely because it sounds elegant. It must be tested against plausible wrong genomes. This is the counter-design principle.

Counter-design asks the designer to state an alternative genome that a competent team might plausibly choose, design briefly under that genome, and show why the resulting system fails the

original gap. The wrong genome should not be a straw man. It should be tempting. It should explain some of the evidence while failing deeper survival conditions.

The validation sequence is:

1. State the chosen genome.
2. State at least one plausible wrong genome.
3. Design briefly under the wrong genome.
4. Identify what the wrong design preserves and what it loses.
5. State survival tests predicted by the chosen genome.
6. Trace those survival tests into ontology and architecture.

A genome is stronger when it rejects more wrong designs without becoming vague. If a proposed genome can justify every possible implementation, it is too weak. If it can justify only one preferred stack, it is too narrow. A good genome constrains identity while leaving room for multiple possible embodiments.

Architectural traceability is the supporting proof. Every major technical decision should be traceable from genome to invariant, ontology, survival test, and engineering body.

Genome element	**Invariant**	**Ontological element**	**Survival test**	**Engineering decision**
Recover memory	Decisions keep rationale.	Decision, rationale, evidence, author, time.	A new contributor can reconstruct why a decision was made.	Decision graph and provenance records.
Expose trust	Claims carry grounds.	Claim, source, confidence, dispute.	A user can inspect the basis of a consequential claim.	Evidence links and confidence states.
Preserve relations	Artifacts remain connected.	Artifact, relation, dependency, version.	Broken links are detected before context is lost.	Durable IDs and graph integrity checks.
Resist decay	Staleness is visible.	Freshness, review, obsolete state.	Old claims cannot masquerade as current.	Decay signals and review workflows.
Surface contradiction	Conflicts become actionable.	Contradiction, tension, resolution.	Incompatible claims are visible at point of use.	Conflict detection and resolution queue.

Traceability prevents post-hoc storytelling only when it is used before and during implementation. If a team adds a graph database and later calls it mycelial, traceability has failed. If the genome requires recoverable relations and the graph is chosen because it protects those relations, traceability has design force.

10 The Four Coherence Statements

A project designed through Psilocybin Design should be expressible in four statements. These statements are not bureaucratic documents. They are the minimum expression of design coherence.

10.1 Statement of Genome

The statement of genome says what the software essentially is, what gap it bridges, and what it must sense, remember, transform, connect, expose, protect, repair, or let decay. If this statement is unclear, the design has no stable identity.

A useful form is:

> This software is essentially *[identity]* for *[actors or environment]*, preserving *[relations or invariants]* so that *[meaningful action]* remains possible under *[pressures]*.

10.2 Statement of Organism

The statement of organism says what kind of digital organism the software becomes and why that organism fits the genome. If this statement is decorative, the biological model has failed. If it is precise, it guides structure, behavior, adaptation, and failure analysis.

10.3 Statement of Ontology

The statement of ontology says what exists inside the organism and how those existences relate. If this statement is weak, the software may still be implemented, but its internal world will be unstable.

10.4 Statement of Survival

The statement of survival says what must remain true for the organism to continue living as itself. If this statement cannot be tested, monitored, reviewed, or protected, the design remains incomplete.

The four statements create a compact discipline. They are simple enough to be written before implementation and strong enough to constrain implementation. They do not replace engineering. They prepare engineering so that the stack can be chosen as embodiment rather than guesswork.

11 Canonical Case: Distributed Research Memory

The canonical case is a distributed research team that loses important decisions across messages, documents, meetings, repositories, experiments, and informal discussion. The apparent problem is simple: the team cannot find what it needs. The obvious solutions are a note-taking tool, a searchable archive, a shared drive, a wiki, or a chat bot. Psilocybin Design begins earlier.

11.1 Observed situation

The team works across multiple tools. Some reasoning happens in meetings. Some in chat. Some in code review. Some in documents. Some in notebooks. Some in informal conversations that later become assumptions. New contributors cannot reconstruct why a direction was chosen. Old contributors remember locally but cannot externalize the whole path. Decisions become detached from reasons. Claims survive after their evidence becomes obsolete. Rejected approaches are rediscovered. Contradictions are found late. Confidence decays silently.

The symptomatic gap is search. The functional gap is fragmented knowledge. The relational gap is broken continuity between decision, rationale, evidence, artifact, and future action. The genomic gap is recoverable research memory under distributed uncertainty.

11.2 Language-game evidence

The team repeatedly asks:

- Where was this decided?
- Which version is current?
- Why did we abandon that approach?
- Who changed this assumption?
- What evidence supported this claim?
- Which artifact depends on this result?
- Is this still true?
- Did anyone resolve the contradiction?
- What should a new contributor read first?

These questions reveal that the team is not only lacking storage. It is lacking durable context, provenance, confidence, and relation.

11.3 Hidden verbs

The hidden verbs are: remember, recover, route, relate, verify, distinguish, preserve, decay, surface, reconcile, and onboard. The verbs are not independent. To remember without recoverability is storage. To verify without provenance is assertion. To route without relation is notification. To decay without visibility is silent loss. To onboard without decision memory is repetition.

11.4 Relations and invariants

Hidden verb	**Required relation**	**Invariant**

Remember	Decision to rationale to artifact to time	Consequential decisions remain connected to their reasons and artifacts.
Verify	Claim to evidence to confidence to authority	Claims expose their grounds and uncertainty.
Route	Signal to context to actor to timing	Relevant context reappears where action occurs.
Distinguish	Artifact to version to current state	Users can tell whether knowledge is current, stale, disputed, or obsolete.
Surface	Contradiction to affected claims to resolution path	Conflicts become visible before they damage later work.
Onboard	Contributor to path to concepts to decisions	New contributors can reconstruct enough memory to act responsibly.

11.5 Genome statement

The genome is not note-taking. It is not search. It is not a wiki. It is not a vector database. The genome is:

> This software is essentially a system for preserving and routing recoverable research memory across distributed contributors, artifacts, decisions, claims, and tools so that research knowledge remains actionable despite time, uncertainty, contradiction, and personnel change.

This statement is strong because it rejects plausible but insufficient designs. A searchable archive may help people find documents but still fail to preserve decisions, rationale, confidence, contradiction, and decay. A chat bot may answer questions but still amplify stale or unsupported claims. A wiki may centralize documentation but still detach knowledge from the living flow of research.

11.6 Counter-design: the note-taking genome

A plausible wrong genome is: "The system is a shared note-taking and search tool for research artifacts." A competent team could build this. It could include folders, full-text search, tags, backlinks, summaries, and templates. It would solve some symptoms. It would also fail the deeper gap.

The note-taking genome fails because it treats memory as stored content rather than recoverable relation. It preserves pages but not necessarily decisions. It indexes text but not necessarily rationale. It stores claims but not necessarily confidence. It records current understanding but not necessarily decay. It allows linking but does not require survival of the link between artifact, evidence, decision, and later action.

Under stress, the wrong design produces familiar failures. New contributors still ask why a decision happened. Old assumptions still appear current. Contradictions remain hidden across pages. Rejected approaches are rediscovered. Search returns documents but not the path of reasoning. The system may be useful, but it is not the organism required by the gap.

11.7 Organism selection

A mycelial organism is appropriate because the software must operate through distributed points of activity rather than a single visible center. It must connect fragments that remain locally produced. It must carry signals across documents, meetings, repositories, and decisions. It must preserve hidden relations without forcing all work into one central surface. It must allow knowledge to reappear where it becomes useful. The mycelial analogy constrains the design because it favors distributed sensing, relation-preserving growth, local context, redundancy, and recovery through connection.

The analogy would be decorative if it merely named a graph database as psilocybin. It has design force only if it changes the ontology, survival conditions, and stack. In this case it does. The organism must sense events from many sources. It must preserve relations beneath visible surfaces. It must route context to places of action. It must survive partial channel loss. It must decay stale knowledge visibly rather than pretending memory is permanent.

11.8 Ontology

The organism must recognize contributors, artifacts, decisions, claims, assumptions, evidence, questions, links, repositories, meetings, experiments, unresolved tensions, contradictions, confidence states, and decay states. These entities are not arbitrary. If a decision is not represented, the system cannot preserve decision memory. If evidence is not related to claims, confidence cannot be inspected. If decay is not represented, stale knowledge can masquerade as current knowledge. If contradiction is not represented, conflict remains hidden until it damages action.

11.9 Survival conditions

The organism survives if:

1. a new contributor can recover the path of a decision;
2. a claim can be traced to evidence, author, time, and confidence;
3. related artifacts remain connected across tools;
4. stale or conflicting knowledge can be surfaced;
5. the loss of one communication channel does not destroy memory;
6. uncertainty remains visible rather than flattened into false certainty;
7. research questions, repositories, and participants can change without destroying continuity.

It dies, in the relevant design sense, if it becomes a pile of searchable content without recoverable relations. It also dies if it becomes an answer engine that hides uncertainty, or a documentation site that freezes living research into static pages.

11.10 Stack implications

Only now can the engineering body be selected. The design may require event ingestion from existing tools, durable identifiers, a graph of relations, provenance records, version history, contradiction detection, decay signals, contextual search, permission rules tied to authority, recovery workflows, and tests for broken links or lost provenance. These are not technical ornaments. They follow from the genome.

Need derived from genome	**Likely body part**	**Reason**
Recover decision memory	Provenance graph	Decisions, evidence, actors, and artifacts must remain connected.
Sense distributed activity	Event ingestion layer	Knowledge appears across tools and must be sensed locally.
Preserve relation across change	Durable identifiers	Links must survive renaming, movement, and tool fragmentation.
Expose stale knowledge	Decay model	Age, review state, and contradiction must affect trust.
Route context into work	Contextual retrieval interface	Memory must reappear where action happens, not only in a separate archive.
Protect authority and confidence	Permission and confidence model	Not all claims, users, or contexts have equal authority.
Detect organism damage	Integrity tests	Broken provenance, orphaned decisions, and stale claims should be visible.

A simple searchable note system might satisfy the apparent requirement while failing the organism. Psilocybin Design changes the design outcome because it changes the identity being served.

12 Boundary Case I: Open-Source Project Governance

A second case shows that the method can transfer beyond research memory. Consider a large open-source project with many contributors, maintainers, issues, pull requests, design discussions, release branches, security reports, and governance decisions. The symptomatic gap may be "too many issues" or "slow review." The functional gap may be coordination. The relational gap is continuity of authority, trust, contribution, and decision across distributed participants.

The language-game includes phrases such as: "Who can approve this?", "Why was this proposal rejected?", "Is this issue still valid?", "Which maintainer owns this area?", "Does this change break the roadmap?", "Was this security concern resolved?", and "What is the status of this contributor's proposal?"

The hidden verbs include authorize, route, review, remember, resolve, defer, trust, escalate, and decay. The relations include contributor to authority, issue to owner, pull request to review state, proposal to rationale, release to risk, security report to confidentiality boundary, and decision to governance rule.

A possible genome is:

> This software is essentially a system for preserving accountable contribution flow across distributed participants, artifacts, authority boundaries, and project memory so that an open-source project can evolve without losing trust, continuity, or governance legitimacy.

A plausible wrong genome is: "The system is an issue triage dashboard." That design may improve visibility but fail to preserve authority, rationale, trust, and governance continuity. The dashboard may show what is open, assigned, or stale, but still fail to answer why a decision was made, who had authority to make it, what rule was applied, and how a contributor can responsibly proceed.

The organism may again be mycelial if the project depends on distributed sensing and relation-preserving flow. It may also require a membrane model, because boundaries matter: public discussion, private security reports, maintainer authority, contributor reputation, release gates, and community norms cannot all be treated as one flat space.

The ontology would include contributors, maintainers, modules, issues, pull requests, proposals, decisions, release branches, authority scopes, governance rules, review states, trust states, security boundaries, and decay states. Survival conditions include: decisions remain traceable to governance rules; stale issues do not appear equivalent to active ones; authority boundaries remain visible; contributor pathways remain legible; security-sensitive relations remain protected; and project memory survives maintainer turnover.

This case shows that Psilocybin Design is not limited to research memory. It applies where software must preserve meaningful relation across distributed activity and environmental change.

13 Boundary Case II: When Psilocybin Design Is Unnecessary

A mature theory should know when not to use itself. Consider a small internal utility that renames files according to a fixed convention, or a static invoice-number formatter, or a simple CRUD form for a stable list of office supplies. The gap is narrow. The environment is stable. The identity is obvious. The survival conditions are ordinary correctness, availability, and maintainability. There is no deep need for organismic reasoning.

A Psilocybin Design analysis could still be forced onto the case. The designer might say that the file renamer has a genome of transformation, a membrane between raw and normalized names, and survival conditions of naming integrity. This would be technically possible but theoretically wasteful. Ordinary engineering would be clearer.

This boundary case matters. Psilocybin Design should not become a universal hammer. Its value appears when the design problem includes identity under pressure: memory across time, coordination across actors, trust under uncertainty, adaptation under change, boundary protection, relation preservation, or survival under partial failure. When those pressures are absent, the method may add language without adding design value.

The scope limit is therefore:

> Use Psilocybin Design when the hard part of the software is not merely performing a function, but remaining the right kind of system while the environment changes.

14 Relation to Earlier Software Thought

Psilocybin Design does not reject earlier software design theories. It depends on the seriousness they gave to different layers of software practice. Its contribution is not to replace them, but to name a prior layer that they do not make central.

Tradition	**Contribution**	**Psilocybin Design adds before it**
Dijkstra	Discipline of program structure and control.	Identity before structure.
Parnas	Decomposition around hidden design decisions.	Genome before modular decomposition.
Brooks	Recognition of essential complexity.	Gap as source of essential identity.
Evans / Domain-Driven Design	Domain language, models, and bounded contexts.	Language-game analysis before domain model.
Fielding	Architecture derived from constraints.	Survival conditions before architectural constraints.
Alexander	Patterns as reusable relations between problem and form.	Genome as identity prior to pattern selection.
Simon	Artificial systems mediating inner structure and outer environment.	Organismic identity as the basis of mediation.
Wittgenstein	Meaning through use in forms of life.	Gap language as evidence for software identity.
Distributed cognition	Cognition as distributed across people, artifacts, and environment.	Software organism as a deliberate preservation of such distributed relations.

A program can be well structured and still be the wrong organism. A system can be modular and still hide the wrong decisions. A domain model can be rich and still miss the genome. An architecture can satisfy constraints and still fail the gap. A stack can be modern and still embody a false identity. Psilocybin Design gives these practices an earlier object of care. It asks them to serve the genome, ontology, and survival conditions of the organism.

The theory is therefore modest in form but large in consequence. It does not promise a silver bullet. It does not claim that all software must be biological in language or structure. It does not replace programming, architecture, domain modeling, testing, verification, operations, or maintenance. It claims that before those practices can be fully justified, the identity of the software must be discovered and stated.

15 Limits and Failure Modes

A theory of design should know how it fails. Psilocybin Design fails in several recognizable ways.

15.1 Metaphor as decoration

The theory fails when biological language becomes ornament. Calling a graph database mycelial, a firewall immune, or a message bus nervous does not produce design knowledge. The organismic model must change what is built, tested, monitored, protected, or removed. If it does not, the language should be discarded.

15.2 Genome asserted instead of extracted

The theory fails when the designer simply declares that the system's genome is memory, trust, coordination, or adaptation. The genome must be supported by gap evidence and language-game analysis. It must explain repeated questions, practices, failures, and workarounds. It must also survive counter-design.

15.3 False genome

A false genome is an identity statement that sounds plausible but preserves the wrong thing. For example, a team may claim that its research system is an archive when the real genome is recoverable decision memory. The archive genome preserves documents. The memory genome preserves relations that make documents actionable.

15.4 Metaphor lock-in

The theory fails when the chosen organism prevents better engineering choices. The biological analogy should constrain design, but it should not become sacred. If the organism no longer clarifies survival, it must be revised or discarded.

15.5 Ontology inflation

The theory fails when ontology becomes a catalogue of everything in the domain. An entity belongs only if the organism must recognize it to preserve its genome. An inflated ontology weakens the system's internal world by making all distinctions appear equally important.

15.6 Survival overfitting

The theory fails when survival tests protect only the designer's initial interpretation rather than the system's real identity under changing conditions. Survival conditions should be reviewed as the environment changes, but the genome should not be revised casually to excuse every new feature request.

15.7 Traceability theater

The theory fails when architecture is justified after the fact. A team may build a system using preferred technologies and later draw arrows back to genome language. This is not traceability. Traceability has force only when it influences decisions before and during implementation.

15.8 Overuse

The theory fails when used on trivial or stable problems where ordinary engineering is enough. A method that cannot say when it is unnecessary becomes suspicious. Psilocybin Design earns trust by limiting its own domain.

16 A Compact Formal Statement

The theory can be stated compactly. Let G be a gap, L the language-game around the gap, V the set of hidden verbs extracted from the language-game, R the relations implied by those verbs, I the invariants derived from the relations, and Γ the genome compressed from the invariants.

$$G \xrightarrow{evidence} L \xrightarrow{interpretation} V \xrightarrow{relation} R \xrightarrow{stability} I \xrightarrow{compression} \Gamma.$$

The genome Γ is acceptable only if it satisfies four conditions.

1. **Explanatory fit:** it explains the repeated language and workarounds around the gap.
2. **Rejection power:** it rejects plausible wrong designs through counter-design.
3. **Survival prediction:** it predicts what the system must preserve, expose, repair, defend, or let decay.
4. **Traceability:** it can be traced into ontology, survival tests, and engineering decisions.

Once Γ is accepted, the designer may select an organismic model O only if the model's survival logic constrains the design. The organism implies an ontology Ω, survival conditions S, death conditions D, and stack decisions B.

$$\Gamma \to O \to \Omega \to S, D \to B.$$

A Psilocybin Design is coherent when the engineering body B can be justified by the survival conditions S, which are grounded in the ontology Ω, which is required by the organism O, which is selected because it fits the genome Γ, which was extracted from the original gap G.

This formalism is intentionally light. It is not a calculus of software. It is a check against premature construction and metaphor drift.

17 Conclusion

Psilocybin Design begins with a simple reversal. Do not begin with the stack. Do not begin with the feature list. Do not begin with architecture. Begin with the gap that makes software necessary.

The gap speaks through language-games. Language-games reveal hidden verbs. Hidden verbs imply relations. Relations produce invariants. Invariants compress into a genome. The genome selects an organism only when an organism's survival logic fits the identity. The organism requires an

ontology. The ontology exposes survival and death conditions. The survival conditions justify the engineering body.

This movement gives software design a prior discipline. It asks what the software must remain before deciding what it should contain. It treats engineering not as the first act of design, but as the embodiment of an identity already discovered.

Psilocybin Design does not make software biological. It gives software design a disciplined way to discover what a system must remain before deciding how it should be built. It asks designers to protect the identity of the software before they construct its body. The result is not metaphorical softness but engineering accountability: every major component should serve the genome, support the ontology, or protect survival.

Software is usually described as something constructed. Psilocybin Design proposes that the deeper act is to bring into existence a digital organism capable of surviving the gap that called it forth.

References

[1] Christopher Alexander, Sara Ishikawa, and Murray Silverstein. *A Pattern Language: Towns, Buildings, Construction*. Oxford University Press, New York, 1977.

[2] Frederick P. Brooks, Jr. No Silver Bullet: Essence and Accidents of Software Engineering. *Computer*, 20(4):10–19, 1987. doi: 10.1109/MC.1987.1663532.

[3] Edsger W. Dijkstra. Letters to the Editor: Go To Statement Considered Harmful. *Communications of the ACM*, 11(3):147–148, 1968. doi: 10.1145/362929.362947.

[4] Eric Evans. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley Professional, Boston, 2003.

[5] Roy Thomas Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. PhD dissertation, University of California, Irvine, 2000. Available at: `https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm`.

[6] Edwin Hutchins. *Cognition in the Wild*. MIT Press, Cambridge, MA, 1995.

[7] Humberto R. Maturana and Francisco J. Varela. *Autopoiesis and Cognition: The Realization of the Living*. D. Reidel Publishing Company, Dordrecht, 1980.

[8] David L. Parnas. On the Criteria To Be Used in Decomposing Systems into Modules. *Communications of the ACM*, 15(12):1053–1058, 1972. doi: 10.1145/361598.361623.

[9] Herbert A. Simon. *The Sciences of the Artificial*. Reissue of the third edition, The MIT Press, Cambridge, MA, 2019. First published 1969; third edition first published 1996.

[10] Lucy Suchman. *Human-Machine Reconfigurations: Plans and Situated Actions*. Cambridge University Press, Cambridge, 2007.

[11] Terry Winograd and Fernando Flores. *Understanding Computers and Cognition: A New Foundation for Design*. Ablex Publishing, Norwood, NJ, 1986.

[12] Ludwig Wittgenstein. *Philosophical Investigations*. Fourth edition. Translated by G. E. M. Anscombe, P. M. S. Hacker, and Joachim Schulte. Revised by P. M. S. Hacker and Joachim Schulte. Wiley-Blackwell, Chichester, 2009. First published 1953.